SYNERGY NETWORK

Remote Procedure Call (RPC) Specification

Canonical developer manual for the Synergy Network RPC surface

Part 1 of 2

Core and Security-Controlled Interfaces

Document	Synergy Network Remote Procedure Call (RPC) Specification
Scope	Canonical developer manual for the Synergy Network RPC surface, with protocol-security-constrained semantics.
Coverage	Core protocol, identity and recovery, AIVM, governance, SXCP, analytics, operations, compatibility, and security-controlled interfaces.

Normative security baseline Methods whose legacy semantics would conflict with the Synergy security architecture are re-scoped here to the secure canonical form. Deployment policy is described through exposure profiles rather than environment nicknames.

How to use this manual

Part 1 establishes the common transport, type, security, and authorization model used by both documents.

This specification is the canonical developer manual for the Synergy Network RPC surface. It merges the uploaded RPC method inventories into a single normalized interface and re-scopes every conflicting surface to the secure canonical form required by the Synergy security architecture.

All state-mutating methods in this manual inherit the standard authorization, nonce, expiry, and simulation requirements unless the method is explicitly identified as operator-profile or authority-plane only.

Common type	Form	Meaning
address / identity	bech32m or canonical string address	Canonical account, validator, contract, or identity address.
bytes32	32-byte hex value	Hashes, identifiers, and fixed-size binary keys.
uint64	unsigned 64-bit integer	Block numbers, timestamps, epochs, nonces, and counts.
uint256	unsigned 256-bit integer	Asset values, fees, balances, and caps.
blockTag	string	Named chain reference such as latest, pending, finalized, or an explicit height/hash form.
AuthorizationSignature	object	Structured authorization envelope bound to chain, selector, nonce, and expiry.

Exposure profile	Intended use
Public read profile	Safe query surfaces intended for wallets, explorers, and public clients.
Authenticated client profile	User-facing state mutation under authorization, nonce, and simulation controls.
Authority plane profile	Signing, guardian, recovery, and account-protection controls.
Operator profile	Administrative or node-local controls protected by strict policy and audit logging.
WebSocket subscription profile	Streaming delivery for event and status channels.

Standard authorization envelope

Authorization is structured, chain-bound, selector-bound, nonce-bound, and expiry-bound. A method that changes state is not authorized by an opaque signature blob.

```
{
  "chain_id": 338639,
  "contract_address": "0x…",
  "function_selector": "0xa9059cbb",
  "nonce": 42,
  "expiry": 1735689600,
  "signature": "0x…"
}
```

Canonical transaction envelope

The canonical transaction envelope is the baseline execution object referenced by transaction, token, staking, governance, bridge, and contract methods.

```
{
  "from": "synw…",
  "to": "sync…",
  "value": 1000000000,
  "data": "0x…",
  "gas_limit": 21000,
  "max_fee": 1000,
  "nonce": 42,
  "chain_id": 338639
}
```

Normative security overrides

The source inventories included several legacy-style surfaces. This manual documents the secure canonical semantics for those surfaces.

Legacy tendency	Canonical Synergy rule
Open-ended approvals	Canonical form uses expiry-bound scoped approvals with amount caps, selectors, and parameter constraints.
Raw message signing	Canonical form uses typed message envelopes; raw hash signing is outside this interface.
Single-step high-value transfer	Canonical form may route transfers into pending windows with watchdog and panic controls.
Bare key import and spending	Canonical form uses factor-bound root authority and tiered keys on the authority plane.
Multicall value reuse	Canonical form preserves per-sub-call manifest review and linear-value semantics.
Cross-chain wildcard authorization	Canonical form rejects chain_id=0 and requires explicit chain binding on authorization envelopes.

Common error model

Code	Meaning
-32600	Invalid Request
-32601	Method not found
-32602	Invalid params
-32000	Generic server error
APPROVAL_EXPIRED	Approval existed but is outside its validity window or was invalidated.
NONCE_ALREADY_USED	Authorization nonce is stale or has already been consumed.
INVALID_CHAIN_ID_ZERO	Authorization chain binding cannot be zero.
SIMULATION_DIVERGENCE	Preflight execution produced unstable or policy-blocked results.

Core Chain and Block State

13 methods

Canonical chain-state and block retrieval methods. These surfaces expose finalized chain views, deterministic block access, and proof-friendly identifiers used throughout the rest of the specification.

Methods: `synergy_blockNumber`, `synergy_getBlockNumber`, `synergy_getChainId`, `synergy_getBlockByNumber`, `synergy_getBlockByHash`, `synergy_getLatestBlock`, `synergy_getBlockRange`, `synergy_getDeterminismDigest`, `synergy_getBlockTransactionCount`, `synergy_getBlockReceipts`, `synergy_getTransactionsInBlock`, `synergy_getTransactionByBlockNumberAndIndex`, `synergy_getTransactionByBlockHashAndIndex`

`synergy_blockNumber`

```
synergy_blockNumber()
```

Get the current block number (height).

Access profile: Public read profile.

Parameters

None.

Returns

number - Current block number

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getBlockNumber`

```
synergy_getBlockNumber()
```

Get the current block number (height). (Alias for synergy_blockNumber)

Access profile: Public read profile.

Parameters

None.

Returns

number - Current block number

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getChainId`

```
synergy_getChainId()
```

Get the chain ID.

Access profile: Public read profile.

Parameters

None.

Returns

number - Chain ID (338639 for network)

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getBlockByNumber`

```
synergy_getBlockByNumber(blockNumber)
```

Get a block by its block number.

Access profile: Public read profile.

Parameters

- **`blockNumber`** *(number)* — The block number to fetch

Returns

Block object or null if not found

Block Object

```
{
  "block_index": 123,
  "timestamp": 1640995200,
  "hash": "...",
  "previous_hash": "...",
  "parent_hash": "...",
  "validator_id": "synv...",
  "validator": "synv...",
  "nonce": 0,
  "tx_count": 10,
  "transactions": [...]
}
```

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getBlockByHash`

```
synergy_getBlockByHash(blockHash)
```

Get a block by its hash.

Access profile: Public read profile.

Parameters

- **`blockHash`** *(string)* — The block hash to fetch

Returns

Block object or null if not found

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getLatestBlock`

```
synergy_getLatestBlock()
```

Get the most recent block.

Access profile: Public read profile.

Parameters

None.

Returns

Block object or null if no blocks exist

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getBlockRange`

```
synergy_getBlockRange(start, end)
```

Get a range of blocks.

Access profile: Public read profile.

Parameters

- **`start`** *(number)* — Starting block number
- **`end`** *(number)* — Ending block number

Returns

Array of block objects

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getDeterminismDigest`

```
synergy_getDeterminismDigest()
```

Get a comprehensive determinism digest for verifying state consistency across nodes.

Access profile: Public read profile.

Parameters

None.

Returns

- Object with:
- block_height: Current block height
- block_hash: Current block hash
- state_root: Combined state root hash
- receipt_hash: Transaction receipt hash
- token_state_hash: Token state hash
- validator_registry_hash: Validator registry hash
- chain_state_hash: Chain state hash

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getBlockTransactionCount`

```
synergy_getBlockTransactionCount(blockNumber)
```

Get the number of transactions in a block.

Access profile: Public read profile.

Parameters

- blockNumber (number) OR blockHash (string) - Block identifier

Returns

number - Transaction count, or null if block not found

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getBlockReceipts`

```
synergy_getBlockReceipts(blockNumber)
```

Get all transaction receipts for a block.

Access profile: Public read profile.

Parameters

- blockNumber (number) OR blockHash (string) - Block identifier

Returns

Array of transaction receipt objects (same format as synergy_getTransactionReceipt), or null if block not found

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getTransactionsInBlock`

```
synergy_getTransactionsInBlock(blockNumber)
```

Get all transactions in a specific block.

Access profile: Public read profile.

Parameters

- **`blockNumber`** *(number)* — Block number

Returns

Array of transaction objects

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getTransactionByBlockNumberAndIndex`

```
synergy_getTransactionByBlockNumberAndIndex(blockNumber, index)
```

Get a transaction by block number and index.

Access profile: Public read profile.

Parameters

- **`blockNumber`** *(number)* — Block number
- **`index`** *(number)* — Transaction index within the block

Returns

Transaction object or null if not found

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getTransactionByBlockHashAndIndex`

```
synergy_getTransactionByBlockHashAndIndex(blockHash, index)
```

Get a transaction by block hash and index.

Access profile: Public read profile.

Parameters

- **`blockHash`** *(string)* — Block hash
- **`index`** *(number)* — Transaction index within the block

Returns

Transaction object or null if not found

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

Transactions, Receipts and Execution

9 methods

Submission, simulation, receipt, and execution lifecycle methods. The secure canonical form requires structured authorization and preflight simulation for state mutation.

Methods: `synergy_simulateTransaction`, `synergy_sendTransaction`, `synergy_getTransactionByHash`, `synergy_getTransactionReceipt`, `synergy_getTransactionCount`, `synergy_getTransactionPool`, `synergy_getPendingTransactions`, `synergy_call`, `synergy_estimateGas`

`synergy_simulateTransaction`

```
synergy_simulateTransaction(transaction)
```

Simulates a state-mutating transaction and returns a deterministic execution preview before submission.

Access profile: Authenticated client profile.

Parameters

- **`transaction`** *(object)* — Canonical transaction envelope including sender, target, value, calldata, fee fields, nonce, and chain binding.

Returns

Simulation result object with asset flows, approvals created or consumed, delegations created or consumed, warnings, and a divergence flag.

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.
- Simulation is expected to surface asset flows, approval creation or consumption, delegation changes, and simulation divergence warnings.
- Wallets SHOULD present the simulation summary to the user before signing any state-mutating intent.

`synergy_sendTransaction`

```
synergy_sendTransaction(transaction, authorization, simulationHash)
```

Submits a state-mutating transaction after successful simulation and structured authorization validation.

Access profile: Authenticated client profile.

Parameters

- **`transaction`** *(object)* — Canonical transaction envelope.
- **`authorization`** *(object)* — AuthorizationSignature bound to the target contract, selector, nonce, expiry, and chain.
- **`simulationHash`** *(string, optional)* — Identifier of a previously accepted simulation result.

Returns

Submission result object with tx_hash, mempool status, and any policy-gate warnings acknowledged by the caller.

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.
- Transactions enter an encrypted mempool on profiles that support threshold-encrypted ordering.

- Consensus rejects authorization envelopes with missing mandatory fields, nonce reuse, chain_id=0, or expired validity windows.

`synergy_getTransactionByHash`

```
synergy_getTransactionByHash(txHash)
```

Get a transaction by its hash. Supports multiple hash formats:
- Full hash: syntxn-...
- Raw hash: a0d53ef9...
- With or without 0x prefix

Access profile: Public read profile.

Parameters

- **`txHash`** *(string)* — Transaction hash

Returns

Transaction object or null if not found

Transaction Object

```
{
  "hash": "syntxn-...",
  "sender": "synw...",
  "receiver": "synw...",
  "from": "synw...",
  "to": "synw...",
  "amount": 1000000000,
  "amount_snrg": 1.0,
  "nonce": 1,
  "gas_price": 1000,
  "gas_limit": 21000,
  "fee": 21000000,
  "timestamp": 1640995200,
  "data": "...",
  "signature_algorithm": "fndsa",
  "signature": "...",
  "status": "confirmed",
  "block_number": 123,
  "transaction_index": 0
}
```

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getTransactionReceipt`

```
synergy_getTransactionReceipt(txHash)
```

Get a transaction receipt with execution details.

Access profile: Public read profile.

Parameters

- **`txHash`** *(string)* — Transaction hash

Returns

- Transaction receipt object with:

- transactionHash: Transaction hash
- transactionIndex: Index in block
- blockHash: Block hash
- blockNumber: Block number
- from: Sender address
- to: Recipient address (or null for contract creation)
- cumulativeGasUsed: Total gas used in block up to this tx
- gasUsed: Gas used by this transaction
- effectiveGasPrice: Actual gas price
- status: "0x1" for success
- logs: Event logs array
- logsBloom: Bloom filter for logs
- contractAddress: Created contract address (if applicable, null otherwise)

Note

```
Returns `null` for pending (unmined) transactions.
```

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getTransactionCount`

```
synergy_getTransactionCount(address, blockTag)
```

Get the transaction count (nonce) for an address.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Address to query
- **`blockTag`** *(string, optional)* — "latest" or "pending" (default: "latest")

Returns

number - Transaction count/nonce

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getTransactionPool`

```
synergy_getTransactionPool()
```

Get all pending transactions in the transaction pool.

Access profile: Public read profile.

Parameters

None.

Returns

Array of pending transaction objects

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getPendingTransactions`

```
synergy_getPendingTransactions(limit, sortBy)
```

Get detailed information about pending transactions with sorting.

Access profile: Public read profile.

Parameters

- **`limit`** *(number, optional)* — Maximum transactions to return (default: 100)
- **`sortBy`** *(string, optional)* — Sort field: "gasPrice", "nonce", or "timestamp" (default: "timestamp")

Returns

Array of pending transaction objects sorted by the specified field

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_call`

```
synergy_call(callObject, blockTag)
```

Executes a read-only local contract call without changing state.

Access profile: Authenticated client profile.

Parameters

- **`callObject`** *(object)* — Read-only call object with from, to, data, and value fields.
- **`blockTag`** *(string, optional)* — State reference used for simulation.

Returns

Read-only execution result object.

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

`synergy_estimateGas`

```
synergy_estimateGas(transaction, from, to, data, …)
```

Estimate the gas required for a transaction.

Access profile: Authenticated client profile.

Parameters

- **`transaction`** *(object)* — Transaction object with:
- from: Sender address (optional)
- to: Recipient address (null for contract deploy)
- data: Transaction data hex (optional)
- value: Value to send (optional)

Returns

number - Estimated gas amount

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

Fees, Logs, Code and Proofs

12 methods

Gas pricing, logs, code inspection, and cryptographic proof surfaces used by wallets, explorers, SDKs, and light clients.

Methods: `synergy_gasPrice, synergy_maxFeePerGas, synergy_maxPriorityFeePerGas, synergy_getFeeHistory, synergy_getLogs, synergy_getCode, synergy_getStorageAt, synergy_getProof, synergy_getFilterLogs, synergy_getFilterChanges, synergy_getStateWithProof, synergy_getAccountNonce`

synergy_gasPrice

```
synergy_gasPrice()
```

Get the current gas price based on recent network utilization.

Access profile: Public read profile.

Parameters

None.

Returns

number - Current gas price in nWei per gas unit

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

synergy_maxFeePerGas

```
synergy_maxFeePerGas()
```

Get the maximum fee per gas.

Access profile: Public read profile.

Parameters

None.

Returns

number - Max fee per gas (2x default gas price)

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

synergy_maxPriorityFeePerGas

```
synergy_maxPriorityFeePerGas()
```

Get the maximum priority fee per gas.

Access profile: Public read profile.

Parameters

None.

Returns

number - Max priority fee per gas (1/4 of default gas price)

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getFeeHistory`

```
synergy_getFeeHistory(blockCount, newestBlock, rewardPercentiles)
```

Get historical gas fee information.

Access profile: Public read profile.

Parameters

- **`blockCount`** *(number, optional)* — Number of blocks to include (default: 10)
- **`newestBlock`** *(string, optional)* — Highest block tag (default: "latest")
- **`rewardPercentiles`** *(array, optional)* — Percentiles to calculate (e.g., [25, 50, 75])

Returns

- Fee history object with:
- baseFeePerGas: Array of base fees per block
- gasUsedRatio: Array of gas utilization ratios per block
- reward: Array of reward arrays at requested percentiles
- oldestBlock: Oldest block number in the result

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getLogs`

```
synergy_getLogs(filter, fromBlock, toBlock, address, …)
```

Get event logs matching filters.

Access profile: Public read profile.

Parameters

- **`filter`** *(object)* — Filter object with:
- **`fromBlock`** *(number, optional)* — Starting block (default: 0)
- **`toBlock`** *(number, optional)* — Ending block (default: latest)
- **`address`** *(string, optional)* — Filter by address
- **`topics`** *(array, optional)* — Event topics to filter
- **`blockHash`** *(string, optional)* — Specific block hash

Returns

- Array of log objects with:
- logIndex: Log index
- transactionIndex: Transaction index in block

- transactionHash: Transaction hash
- blockHash: Block hash
- blockNumber: Block number
- address: Emitting address
- data: Log data
- topics: Log topics
- removed: Whether log was removed (always false)

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getCode`

```
synergy_getCode(address)
```

Get the code (smart contract bytecode) stored at an address.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Contract address

Returns

string - Contract bytecode (hex) or "0x" if not a contract

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getStorageAt`

```
synergy_getStorageAt(address, position, blockTag)
```

Get the value from a storage position of a contract/account.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Contract/account address
- **`position`** *(string)* — Storage slot position (hex)
- **`blockTag`** *(string, optional)* — "latest", "pending", or block number

Returns

string - Storage value (32 bytes, hex-encoded with 0x prefix)

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getProof`

```
synergy_getProof(address, storageKeys, blockTag)
```

Returns an account or storage proof using the same state-root anchored proof model as synergy_getStateWithProof.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Account address.
- **`storageKeys`** *(array)* — Storage keys to prove.
- **`blockTag`** *(string, optional)* — Finalized block reference.

Returns

Proof object containing the account proof and storage proofs.

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.
- Clients that make security-sensitive decisions from this response SHOULD verify the proof locally against a finalized header.

`synergy_getFilterLogs`

```
synergy_getFilterLogs(filterId)
```

Returns logs associated with a registered poll-based filter.

Access profile: Public read profile.

Parameters

- **`filterId`** *(string)* — Filter ID

Returns

Array of log objects

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.
- Use the subscription interfaces when push delivery is preferred or when filter registration is not enabled.

`synergy_getFilterChanges`

```
synergy_getFilterChanges(filterId)
```

Returns incremental log changes associated with a registered poll-based filter.

Access profile: Public read profile.

Parameters

- **`filterId`** *(string)* — Filter ID

Returns

Array of new log objects since last poll

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.
- Use the subscription interfaces when push delivery is preferred or when filter registration is not enabled.

`synergy_getStateWithProof`

```
synergy_getStateWithProof(address, storageKeys, blockTag)
```

Returns account or storage state together with a cryptographic proof anchored to the finalized state root.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Account or contract address.
- **`storageKeys`** *(array, optional)* — Storage keys to include in the proof.
- **`blockTag`** *(string, optional)* — Finalized block reference.

Returns

State-with-proof object containing the requested values and the Merkle proof payload.

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.
- Clients that make security-sensitive decisions from this response SHOULD verify the proof locally against a finalized header.

`synergy_getAccountNonce`

```
synergy_getAccountNonce(address)
```

Returns the canonical authorization nonce tracked by consensus for an account.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Account address or UMA identity address.

Returns

number - Current global authorization nonce.

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

Validators, Clusters and Consensus

19 methods

Validator registry, cluster composition, performance, reward, and consensus health interfaces.

Methods: `synergy_getValidators`, `synergy_getValidator`, `synergy_getTopValidators`, `synergy_registerValidator`, `synergy_approveValidator`, `synergy_slashValidator`, `synergy_getValidatorStats`, `synergy_getValidatorActivity`, `synergy_getSynergyScoreBreakdown`, `synergy_getBlockValidationStatus`, `synergy_getValidatorByCluster`, `synergy_getValidatorRewards`, `synergy_getValidatorPerformance`, `synergy_getValidatorQueue`, `synergy_requestValidatorExit`, `synergy_getValidatorSlashingHistory`, `synergy_getClusterInfo`, `synergy_getClusterRewards`, `synergy_proposeClusterChange`

`synergy_getValidators`

```
synergy_getValidators()
```

Get all active validators.

Access profile: Public read profile.

Parameters

None.

Returns

- Array of validator objects with:
- address: Validator address
- public_key: Validator public key
- name: Validator name
- stake_amount: Staked amount
- synergy_score: Current synergy score
- uptime_percentage: Uptime percentage
- total_blocks_produced: Total blocks produced
- cluster_id: Cluster ID
- last_active: Last activity timestamp
- status: Validator status

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getValidator`

```
synergy_getValidator(address)
```

Get a specific validator by address.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Validator address

Returns

Validator object or null if not found

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getTopValidators`

```
synergy_getTopValidators(count)
```

Get top validators by rank.

Access profile: Public read profile.

Parameters

- **`count`** *(number, optional)* — Number of validators to return (default: 10)

Returns

Array of top validator objects

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_registerValidator`

```
synergy_registerValidator(address, public_key, name, stake_amount)
```

Register a new validator.

Access profile: Authenticated client profile.

Parameters

- **`address`** *(string)* — Validator address
- **`public_key`** *(string)* — Validator public key
- **`name`** *(string)* — Validator name
- **`stake_amount`** *(number)* — Initial stake amount

Returns

```
json
{
  "success": true,
  "message": "Validator registered successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

`synergy_approveValidator`

```
synergy_approveValidator(address)
```

Approve a validator registration.

Access profile: Authenticated client profile.

Parameters

- **`address`** *(string)* — Validator address

Returns

```
json
{
  "success": true,
  "message": "Validator approved successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

`synergy_slashValidator`

```
synergy_slashValidator(address, reason)
```

Slash a validator (penalize for misbehavior).

Access profile: Authenticated client profile.

Parameters

- **`address`** *(string)* — Validator address
- **`reason`** *(string)* — Reason for slashing

Returns

```
json
{
  "success": true,
  "message": "Validator slashed successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

`synergy_getValidatorStats`

```
synergy_getValidatorStats()
```

Get validator statistics.

Access profile: Public read profile.

Parameters

None.

Returns

- Stats object with:
- total_validators: Total number of validators
- active_validators: Array of active validators
- top_validators: Array of top validators
- epoch_rewards: Epoch rewards information

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getValidatorActivity`

```
synergy_getValidatorActivity()
```

Get validator activity information.

Access profile: Public read profile.

Parameters

None.

Returns

- Activity object with:

- validators: Array of validator activity objects
- total_active: Total active validators
- average_synergy_score: Average synergy score

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getSynergyScoreBreakdown`

```
synergy_getSynergyScoreBreakdown(address)
```

Get detailed Synergy Score components for a validator.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Validator address

Returns

- Object with:
- address: Validator address
- total_score: Current normalized Synergy Score
- components: Synergy score component object containing:
- stake_weight: Stake weight component
- reputation: Reputation component
- contribution_index: Contribution index
- cartelization_penalty: Cartelization penalty
- normalized_score: Normalized score
- last_updated: Last update timestamp

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getBlockValidationStatus`

```
synergy_getBlockValidationStatus()
```

Get block validation status.

Access profile: Public read profile.

Parameters

None.

Returns

- Validation status object with:
- current_block_height: Current block height
- recent_blocks: Array of recent block validation info
- validation_queue: Pending validation queue (empty array)
- active_validators: Number of active validators
- total_validators: Total number of validators
- cluster_info: Cluster information with:
- active_clusters: Number of active clusters

- total_stake: Total stake across clusters

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getValidatorByCluster`

```
synergy_getValidatorByCluster(clusterId)
```

Get all validators in a specific cluster.

Access profile: Public read profile.

Parameters

- **`clusterId`** *(number)* — Cluster ID

Returns

Array of validator objects filtered by cluster membership

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getValidatorRewards`

```
synergy_getValidatorRewards(address, fromEpoch, toEpoch)
```

Get rewards earned by a validator over time.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Validator address
- **`fromEpoch`** *(number, optional)* — Starting epoch
- **`toEpoch`** *(number, optional)* — Ending epoch

Returns

- Object with:
- address: Validator address
- totalBlocksProduced: Total blocks produced
- rewards: Array of reward objects (blockNumber, amount, type, timestamp)
- totalRewards: Sum of all rewards

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getValidatorPerformance`

```
synergy_getValidatorPerformance(address)
```

Get detailed performance metrics for a validator.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Validator address

Returns

- Performance object with:
- attestationSuccessRate: Uptime percentage
- blockProposalSuccessRate: Block proposal success rate
- averageInclusionDelay: Average block time
- missedAttestations: Missed blocks count
- orphanedBlocks: Orphaned blocks count
- effectiveBalance: Current stake amount
- totalBlocksProduced: Blocks produced count
- synergyScore: Current synergy score
- reputationScore: Reputation score
- collaborationScore: Collaboration score
- uptime: Uptime percentage

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getValidatorQueue`

```
synergy_getValidatorQueue()
```

Get the validator activation/deactivation queue status.

Access profile: Public read profile.

Parameters

None.

Returns

- Queue object with:
- activationQueue: Array of pending validator registrations
- activationQueueLength: Number of validators waiting
- exitQueue: Array of jailed validators
- exitQueueLength: Number of validators exiting
- estimatedActivationTime: Estimated activation timestamp
- estimatedExitTime: Estimated exit timestamp

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_requestValidatorExit`

```
synergy_requestValidatorExit(address, signature)
```

Request a validator to exit (initiate unstaking period).

Access profile: Authenticated client profile.

Parameters

- **`address`** *(string)* — Validator address
- **`signature`** *(string)* — Signed exit request

Returns

```
json
{
  "success": true,
  "message": "Validator exit requested",
  "validatorAddress": "synv...",
  "currentEpoch": 10,
  "exitEpoch": 12,
  "withdrawalAvailableAt": 1640995200
}
```

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

`synergy_getValidatorSlashingHistory`

```
synergy_getValidatorSlashingHistory(address)
```

Get slashing history for a validator.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Validator address

Returns

- Object with:
- address: Validator address
- slashingEvents: Array of slashing event objects
- totalPenalties: Total penalty amount
- doubleSignCount: Number of double-sign infractions

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getClusterInfo`

```
synergy_getClusterInfo(clusterId)
```

Get detailed information about a validator cluster.

Access profile: Public read profile.

Parameters

- **`clusterId`** *(number)* — Cluster ID

Returns

- Cluster info object with:
- clusterId: Cluster ID
- address: Cluster address
- validators: Array of validator detail objects

- validatorCount: Number of validators
- totalStake: Total stake in cluster
- averageSynergyScore: Average synergy score
- createdAt: Creation timestamp
- lastRotation: Last rotation timestamp
- group: Cluster group number

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getClusterRewards`

```
synergy_getClusterRewards(clusterId, epoch)
```

Get rewards distribution for a cluster.

Access profile: Public read profile.

Parameters

- **`clusterId`** *(number)* — Cluster ID
- **`epoch`** *(number, optional)* — Specific epoch (default: current)

Returns

- Object with:
- clusterId: Cluster ID
- epoch: Epoch number
- totalRewards: Total rewards for cluster
- distributions: Array of per-validator reward objects

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_proposeClusterChange`

```
synergy_proposeClusterChange(clusterId, proposal, proposer)
```

Propose a change to cluster composition.

Access profile: Authenticated client profile.

Parameters

- **`clusterId`** *(number)* — Cluster ID
- **`proposal`** *(object)* — Proposal details
- **`proposer`** *(string)* — Proposer address (must be a registered validator)

Returns

```
json
{
  "success": true,
  "proposalId": "prop_1_1640995200",
  "clusterId": 1,
  "proposer": "synv...",
  "votingEndsAt": 1641081600
```

```
}
```

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

Tokens, Assets and Approvals

15 methods

Native asset, token, transfer, and approval surfaces. Approval semantics are constrained by the Synergy security model rather than open-ended allowance patterns.

Methods: `synergy_getBalance, synergy_getTokenBalance, synergy_getTokens, synergy_getAllBalances, synergy_sendTokens, synergy_createToken, synergy_mintTokens, synergy_burnTokens, synergy_transferTokens, synergy_getTokenStats, synergy_getTransferHistory, synergy_createApproval, synergy_getActiveApprovals, synergy_getApprovalHistory, synergy_revokeAllApprovals`

`synergy_getBalance`

```
synergy_getBalance(address)
```

Get the SNRG balance for an address (standardized method).

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Address to query

Returns

number - Balance in nWei

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getTokenBalance`

```
synergy_getTokenBalance(address, token)
```

Get token balance for an address.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Address to check
- **`token`** *(string)* — Token symbol (e.g., "SNRG")

Returns

Balance in nWei (nano-wei, smallest unit)

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getTokens`

```
synergy_getTokens()
```

Get all tokens on the network.

Access profile: Public read profile.

Parameters

None.

Returns

- Array of token objects with:
- symbol: Token symbol
- name: Token name
- decimals: Number of decimals
- total_supply: Total supply
- max_supply: Maximum supply
- mintable: Whether token is mintable
- burnable: Whether token is burnable
- created_at: Creation timestamp
- creator: Creator address

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getAllBalances`

```
synergy_getAllBalances(address)
```

Get all token balances for an address.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Address to check

Returns

Object mapping token symbols to balances (in nWei)

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_sendTokens`

```
synergy_sendTokens(from, to, token_symbol, amount)
```

Send tokens from one address to another.

Access profile: Authenticated client profile.

Parameters

- **`from`** *(string)* — Sender address
- **`to`** *(string)* — Recipient address
- **`token_symbol`** *(string)* — Token symbol (e.g., "SNRG")

- **amount** *(number)* — **Amount in SNRG** (will be converted to nWei internally)

Returns

```
json
{
  "success": true,
  "tx_hash": "syntxn-...",
  "transaction": {...},
  "message": "Transaction submitted"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

synergy_createToken

```
synergy_createToken(symbol, name, decimals, total_supply, …)
```

Create a new token.

Access profile: Authenticated client profile.

Parameters

- **symbol** *(string)* — Token symbol
- **name** *(string)* — Token name
- **decimals** *(number)* — Number of decimals
- **total_supply** *(number)* — Total supply
- **creator** *(string)* — Creator address

Returns

```
json
{
  "success": true,
  "message": "Token created successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

synergy_mintTokens

```
synergy_mintTokens(to, token_symbol, amount)
```

Mint new tokens.

Access profile: Authenticated client profile.

Parameters

- **`to`** *(string)* — Recipient address
- **`token_symbol`** *(string)* — Token symbol
- **`amount`** *(number)* — Amount to mint

Returns

```
json
{
  "success": true,
  "message": "Tokens minted successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

`synergy_burnTokens`

```
synergy_burnTokens(from, token_symbol, amount)
```

Burn tokens.

Access profile: Authenticated client profile.

Parameters

- **`from`** *(string)* — Address to burn from
- **`token_symbol`** *(string)* — Token symbol
- **`amount`** *(number)* — Amount to burn

Returns

```
json
{
  "success": true,
  "message": "Tokens burned successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

synergy_transferTokens

```
synergy_transferTokens(from, to, token_symbol, amount)
```

Transfer tokens (low-level method).

Access profile: Authenticated client profile.

Parameters

- **from** *(string)* — Sender address
- **to** *(string)* — Recipient address
- **token_symbol** *(string)* — Token symbol
- **amount** *(number)* — Amount to transfer

Returns

```
json
{
  "success": true,
  "message": "Tokens transferred successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

synergy_getTokenStats

```
synergy_getTokenStats()
```

Get token statistics.

Access profile: Public read profile.

Parameters

None.

Returns

- Array of token stat objects with:
- symbol: Token symbol
- name: Token name
- total_supply: Total supply
- total_staked: Total staked amount
- holders: Number of holders

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

synergy_getTransferHistory

```
synergy_getTransferHistory(address, limit)
```

Get transfer history for an address.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Address to check
- **`limit`** *(number, optional)* — Maximum number of transfers to return (default: 50)

Returns

- Array of transfer objects with:
- from: Sender address
- to: Recipient address
- token_symbol: Token symbol
- amount: Amount transferred
- fee: Transaction fee
- timestamp: Transfer timestamp
- tx_hash: Transaction hash
- block_height: Block height

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_createApproval`

```
synergy_createApproval(approval, authorization)
```

Creates a scoped approval record enforced at the consensus layer.

Access profile: Authenticated client profile.

Parameters

- **`approval`** *(object)* — Approval object with spender, amount_cap, expiry_timestamp, function_selector, and parameter_constraints.
- **`authorization`** *(object)* — AuthorizationSignature bound to the approval operation.

Returns

Object containing the approval identifier, creation status, and effective scope metadata.

Approval Object

```
{
  "spender": "sync...",
  "amount_cap": 1000000000,
  "expiry_timestamp": 1735689600,
  "function_selector": "0xa9059cbb",
  "parameter_constraints": {"token":"SNRG"}
}
```

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.
- Approvals are scoped by spender, amount_cap, expiry_timestamp, function_selector, and parameter_constraints.

- Approvals without expiry are invalid; broad or dormant approvals are outside the Synergy security model.

`synergy_getActiveApprovals`

```
synergy_getActiveApprovals(owner)
```

Returns all active approvals for an owner account, including residual caps and effective expiry.

Access profile: Public read profile.

Parameters

- **`owner`** *(string)* — Owner address or UMA identity address.

Returns

Array of active approval objects.

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getApprovalHistory`

```
synergy_getApprovalHistory(owner, filters)
```

Returns the full protocol audit history for approvals created, used, expired, or revoked by an owner account.

Access profile: Public read profile.

Parameters

- **`owner`** *(string)* — Owner address or UMA identity address.
- **`filters`** *(object, optional)* — Filter by spender, status, token, or time window.

Returns

Array of approval event records ordered by block height and event index.

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.
- The history is sourced from the consensus-level approval audit log rather than contract-emitted events alone.

`synergy_revokeAllApprovals`

```
synergy_revokeAllApprovals(owner, authorization)
```

Atomically revokes all approvals associated with an owner account in a single consensus transition.

Access profile: Authenticated client profile.

Parameters

- **`owner`** *(string)* — Owner address or UMA identity address.
- **`authorization`** *(object)* — AuthorizationSignature bound to the revoke-all operation.

Returns

Object with revoked_count and final status.

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.
- Revocation is atomic: either all tracked approvals are invalidated in one state transition or none are.

Staking, Delegation and Rewards

12 methods

Delegation, staking state, APY, reward accounting, claims, and validator-exit interfaces.

Methods: `synergy_stakeTokens, synergy_stakeTokensDirect, synergy_unstakeTokens, synergy_getStakedBalance, synergy_getStakingInfo, synergy_getStakingRewards, synergy_getStakingAPY, synergy_getDelegatedStakes, synergy_getDelegators, synergy_claimRewards, synergy_getRewardsProjection, synergy_getUnstakingPeriod`

synergy_stakeTokens

```
synergy_stakeTokens(staker, validator, token_symbol, amount)
```

Create a staking transaction.

Access profile: Authenticated client profile.

Parameters

- **`staker`** *(string)* — Staker address
- **`validator`** *(string)* — Validator address
- **`token_symbol`** *(string)* — Token symbol
- **`amount`** *(number)* — **Amount in SNRG** (will be converted to nWei)

Returns

```
json
{
  "success": true,
  "transaction": {...},
  "message": "Staking transaction created successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

synergy_stakeTokensDirect

```
synergy_stakeTokensDirect(staker, validator, token_symbol, amount)
```

Stake tokens directly (without creating a transaction).

Access profile: Authenticated client profile.

Parameters

- **`staker`** *(string)* — Staker address
- **`validator`** *(string)* — Validator address
- **`token_symbol`** *(string)* — Token symbol
- **`amount`** *(number)* — **Amount in SNRG** (will be converted to nWei)

Returns

```
json
{
  "success": true,
  "message": "Tokens staked successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

`synergy_unstakeTokens`

```
synergy_unstakeTokens(staker, validator, token_symbol, amount)
```

Unstake tokens.

Access profile: Authenticated client profile.

Parameters

- **`staker`** *(string)* — Staker address
- **`validator`** *(string)* — Validator address
- **`token_symbol`** *(string)* — Token symbol
- **`amount`** *(number)* — Amount to unstake

Returns

```
json
{
  "success": true,
  "message": "Tokens unstaked successfully"
}
`
```

or error object

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

`synergy_getStakedBalance`

```
synergy_getStakedBalance(address, token_symbol)
```

Get staked balance for an address.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Address to check
- **`token_symbol`** *(string)* — Token symbol

Returns

```
json
{
  "balance": 1000000
}
```

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getStakingInfo`

```
synergy_getStakingInfo(address)
```

Get staking information for an address.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Address to check

Returns

- Array of staking info objects with:
- validator: Validator address
- staked_amount: Staked amount
- rewards_earned: Rewards earned
- staking_timestamp: Staking timestamp

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getStakingRewards`

```
synergy_getStakingRewards(address, validator)
```

Get staking rewards for an address.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Staker address
- **`validator`** *(string, optional)* — Filter by specific validator

Returns

- Object with:
- address: Staker address

- rewards: Array of reward objects (validator, stakedAmount, rewardsEarned, stakingStart, isActive)
- totalRewardsEarned: Sum of all rewards earned

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getStakingAPY`

```
synergy_getStakingAPY(validator)
```

Get current staking APY information.

Access profile: Public read profile.

Parameters

- **`validator`** *(string, optional)* — Specific validator for validator-specific APY

Returns

- APY object with:
- currentAPY: Current annual percentage yield (capped at 20%)
- averageAPY: Average APY
- networkStakingRate: Percentage of total supply staked
- totalStaked: Total staked amount
- totalSupply: Total token supply
- baseAPY: Base annual yield (5%)
- validatorAPY: Validator-specific APY (if validator specified)
- validatorSynergyScore: Validator's synergy score (if validator specified)

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getDelegatedStakes`

```
synergy_getDelegatedStakes(address)
```

Get all delegated stakes for a staker.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Staker address

Returns

- Object with:
- address: Staker address
- delegations: Array of active delegation objects (validator, amount, rewardsEarned, delegatedAt)
- totalDelegated: Total amount delegated

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

synergy_getDelegators

```
synergy_getDelegators(validator, limit)
```

Get all delegators for a validator.

Access profile: Public read profile.

Parameters

- **validator** *(string)* — Validator address
- **limit** *(number, optional)* — Maximum results (default: 100)

Returns

- Object with:
- validator: Validator address
- delegators: Array of delegator objects sorted by amount (descending)
- totalDelegators: Number of delegators

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

synergy_claimRewards

```
synergy_claimRewards(staker, validator)
```

Claim accumulated staking rewards.

Access profile: Authenticated client profile.

Parameters

- **staker** *(string)* — Staker address
- **validator** *(string, optional)* — Specific validator (claims all if not specified)

Returns

```
json
{
  "success": true,
  "claimedAmount": 1000000,
  "stakerAddress": "synw...",
  "message": "Rewards claimed successfully"
}
```

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

synergy_getRewardsProjection

```
synergy_getRewardsProjection(address, amount, duration, validator)
```

Project future staking rewards.

Access profile: Public read profile.

Parameters

- **`address`** *(string)* — Staker address
- **`amount`** *(number)* — Stake amount to project
- **`duration`** *(number)* — Duration in days
- **`validator`** *(string, optional)* — Specific validator

Returns

- Projection object with:
- stakeAmount: Input stake amount
- durationDays: Input duration
- estimatedAPY: Calculated APY
- projectedReward: Estimated reward over period
- projectedTotal: Stake + projected reward

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

`synergy_getUnstakingPeriod`

```
synergy_getUnstakingPeriod()
```

Get information about the unstaking period.

Access profile: Public read profile.

Parameters

None.

Returns

- Object with:
- unstakingPeriodDays: 7 days
- unstakingPeriodSeconds: 604800 seconds
- currentQueueLength: Current unstaking queue length
- estimatedWithdrawalTime: Estimated withdrawal timestamp

Security and execution rules

- Read-only method. It does not change consensus state.
- Deployments may expose this surface on public read profiles unless narrowed by local policy.

Identity, Wallet, Recovery and Delegation Control

22 methods

Identity, SynID resolution, wallet authority, recovery, guardian, pending-transfer, and delegation-control surfaces. These methods belong on authenticated authority planes.

Methods: `synergy_resolveSynID`, `synergy_getAddressBook`, `synergy_createWallet`, `synergy_getWallet`, `synergy_createWalletFromKeypair`, `synergy_getAllWallets`, `synergy_signTransaction`, `synergy_signMessage`, `synergy_verifyMessage`, `synergy_getEncryptionKey`, `synergy_rotateKeys`, `synergy_getActiveDelegations`, `synergy_revokeDelegation`, `synergy_initiateRecovery`, `synergy_confirmRecovery`, `synergy_getGuardians`, `synergy_verifyCurrentAuthKey`, `synergy_getPendingGuardianNotifications`, `synergy_getPendingTransfers`, `synergy_cancelPendingTransfer`, `synergy_freezeAccount`, `synergy_getSecurityAlerts`

synergy_resolveSynID

```
synergy_resolveSynID(synId)
```

Resolves a human-readable SynID to its canonical account address and identity metadata.

Access profile: Authenticated client profile.

Parameters

- **synId** *(string)* — Registered SynID.

Returns

Resolution object containing the target address, canonical SynID, and registration metadata.

Security and execution rules

- State-mutating method. Successful execution is bound to the standard authorization envelope and consensus nonce tracking.
- The caller SHOULD obtain a preflight result from synergy_simulateTransaction or an equivalent domain-specific simulation step before submission.
- Off-chain signatures do not authorize state mutation on the Synergy Network RPC surface.

synergy_getAddressBook

```
synergy_getAddressBook(owner)
```

Returns the consensus-backed address book entries associated with an owner account.

Access profile: Authority plane profile.

Parameters

- **owner** *(string)* — Owner address or UMA identity address.

Returns

Array of address book entries with SynID, address, labels, and verification status.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.

synergy_createWallet

```
synergy_createWallet()
```

Creates a wallet identity and derives the tiered authority keys used by the Synergy wallet architecture.

Access profile: Authority plane profile.

Parameters

None.

Returns

Wallet creation result containing the account address, public authentication material, and policy bootstrap metadata.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.

- Raw hash signing and single-factor spending authority are outside the Synergy security model.

`synergy_getWallet`

```
synergy_getWallet(address)
```

Get wallet information.

Access profile: Authority plane profile.

Parameters

- **`address`** *(string)* — Wallet address

Returns

Wallet object or null if not found

Wallet Object

```
{
  "address": "synw...",
  "public_key": "...",
  "balances": {
    "SNRG": 1000000
  },
  "nonce": 0
}
```

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.

`synergy_createWalletFromKeypair`

```
synergy_createWalletFromKeypair(public_key, key_material_envelope, second_factor_proof)
```

Imports wallet authority from externally generated key material under the factor-bound root-authority model.

Access profile: Authority plane profile.

Parameters

- **`public_key`** *(string)* — Public authentication key.
- **`key_material_envelope`** *(string)* — Encrypted import bundle for private signing material.
- **`second_factor_proof`** *(string)* — Independent factor proof required to activate spending authority.

Returns

Wallet import result object with the derived account address and key-tier metadata.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- Import flows remain subject to factor-bound root authority; importing bare private key material does not bypass second-factor requirements.

`synergy_getAllWallets`

```
synergy_getAllWallets()
```

Get all wallets.

Access profile: Authority plane profile.

Parameters

None.

Returns

Array of wallet objects

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.

`synergy_signTransaction`

```
synergy_signTransaction(address, transaction, policyContext)
```

Produces a structured AuthorizationSignature for a transaction intent within the authority plane.

Access profile: Authority plane profile.

Parameters

- **`address`** *(string)* — Signing identity or account address.
- **`transaction`** *(object)* — Canonical transaction envelope to authorize.
- **`policyContext`** *(object, optional)* — Policy-engine context used for tiering, manifests, and guardian routing.

Returns

Authorization object containing chain binding, nonce, expiry, signature bytes, and the signed transaction intent.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- The result is a structured authorization object, not an unrestricted raw signature blob.

`synergy_signMessage`

```
synergy_signMessage(address, message)
```

Signs a typed message envelope. Raw hash signing is outside the Synergy security model.

Access profile: Authority plane profile.

Parameters

- **`address`** *(string)* — Signing identity or account address.
- **`message`** *(object)* — Typed message envelope with domain, schema, payload, nonce, and expiry.

Returns

Structured message-signature object.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- The signed payload is typed and domain-separated. Opaque hash signing is not part of this interface.

`synergy_verifyMessage`

```
synergy_verifyMessage(address, message, signature)
```

Verifies a typed message-signature envelope against the current account authorization state.

Access profile: Authority plane profile.

Parameters

- **`address`** *(string)* — Claimed signer address.
- **`message`** *(object)* — Typed message envelope.
- **`signature`** *(object)* — Structured message-signature envelope.

Returns

Verification result object with validity, signer identity, and scope metadata.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- Verification is performed against the typed message envelope and the current authorization state.

`synergy_getEncryptionKey`

```
synergy_getEncryptionKey(keyType)
```

Returns wrapped public encryption material used for secure communication and never exposes private key material.

Access profile: Authority plane profile.

Parameters

- **`keyType`** *(string)* — Type of key needed

Returns

Encrypted key object

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.

`synergy_rotateKeys`

```
synergy_rotateKeys(account, newAuthKey, authorization)
```

Rotates the current authentication key without changing the account address or SynID.

Access profile: Authority plane profile.

Parameters

- **`account`** *(string)* — Account address or UMA identity address.
- **`newAuthKey`** *(string)* — Replacement authentication key or key bundle hash.
- **`authorization`** *(object)* — High-tier authority envelope.

Returns

Key rotation result object with the new auth key hash and revocation summary.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- Key rotation preserves the account address and identity binding.
- Deployments SHOULD invalidate active delegations and stale approval authority derived from the previous key as part of the rotation workflow.

`synergy_getActiveDelegations`

```
synergy_getActiveDelegations(account)
```

Returns active execution delegations registered for an account.

Access profile: Authority plane profile.

Parameters

- **`account`** *(string)* — Delegator account address or UMA identity address.

Returns

Array of delegation registry records.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- Delegation records represent execution-authority changes and SHOULD be surfaced prominently to account owners.

`synergy_revokeDelegation`

```
synergy_revokeDelegation(delegationId, authorization)
```

Revokes an active execution delegation.

Access profile: Authority plane profile.

Parameters

- **`delegationId`** *(string)* — Delegation identifier.
- **`authorization`** *(object)* — AuthorizationSignature or guardian-approved authority envelope.

Returns

Object with revocation status and affected scope metadata.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.

- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- Delegation revocation SHOULD take effect before any further delegated execution is admitted.

`synergy_initiateRecovery`

```
synergy_initiateRecovery(account, newAuthKey, guardianProofs)
```

Initiates guardian-mediated account recovery.

Access profile: Authority plane profile.

Parameters

- **`account`** *(string)* — Account address or UMA identity address.
- **`newAuthKey`** *(string)* — Replacement authentication key or key bundle hash.
- **`guardianProofs`** *(array)* — Guardian quorum proofs required by the recovery policy.

Returns

Recovery workflow object with initiation status, delay window, and confirmation eligibility time.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- Recovery requires guardian quorum proofs and is subject to the mandatory recovery delay enforced by protocol policy.

`synergy_confirmRecovery`

```
synergy_confirmRecovery(recoveryId, guardianProofs)
```

Finalizes a previously initiated guardian recovery after the mandatory delay has elapsed.

Access profile: Authority plane profile.

Parameters

- **`recoveryId`** *(string)* — Recovery workflow identifier.
- **`guardianProofs`** *(array, optional)* — Final confirmation proofs when required by policy.

Returns

Object describing the finalized authentication key and recovery completion state.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- Confirmation succeeds only after the recovery delay has elapsed and the workflow remains valid.

`synergy_getGuardians`

```
synergy_getGuardians(account)
```

Returns the guardian registry for an account, including quorum threshold and modification timing.

Access profile: Authority plane profile.

Parameters

- **`account`** *(string)* — Account address or UMA identity address.

Returns

Guardian registry object.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.

`synergy_verifyCurrentAuthKey`

```
synergy_verifyCurrentAuthKey(account)
```

Verifies the currently active authentication key hash bound to an identity without changing the account address.

Access profile: Authority plane profile.

Parameters

- **`account`** *(string)* — Account address or UMA identity address.

Returns

Object containing the current authentication key hash and rotation history summary.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.

`synergy_getPendingGuardianNotifications`

```
synergy_getPendingGuardianNotifications(guardian)
```

Returns guardian notifications raised by high-value pending transfers, delegation changes, or recovery events.

Access profile: Authority plane profile.

Parameters

- **`guardian`** *(string)* — Guardian account address or UMA identity address.

Returns

Array of guardian notification objects.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.

`synergy_getPendingTransfers`

```
synergy_getPendingTransfers(account)
```

Returns pending transfers currently in the protocol delay window for an account.

Access profile: Authority plane profile.

Parameters

- **`account`** *(string)* — Account address or UMA identity address.

Returns

Array of pending transfer records including release time, cancelability, and recipient data.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- Pending transfers exist because the protocol can hold higher-risk transfers in a cancellation window before release.

`synergy_cancelPendingTransfer`

```
synergy_cancelPendingTransfer(transferId, authorization)
```

Cancels a transfer that is still in its protocol-enforced pending window.

Access profile: Authority plane profile.

Parameters

- **`transferId`** *(string)* — Pending transfer identifier.
- **`authorization`** *(object)* — Owner or watchdog authorization envelope.

Returns

Object with the cancellation result and restored balance metadata.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- Owner or watchdog authority may cancel a pending transfer while the delay window remains open.

`synergy_freezeAccount`

```
synergy_freezeAccount(account, authorization, duration)
```

Freezes outgoing transfers from an account through the panic-freeze control path.

Access profile: Authority plane profile.

Parameters

- **`account`** *(string)* — Account address or UMA identity address.
- **`authorization`** *(object)* — Panic-key authorization envelope.
- **`duration`** *(number, optional)* — Freeze duration override when permitted by policy.

Returns

Object describing the freeze window and effective state.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.
- This surface is intended for panic-freeze controls and SHOULD have priority over ordinary outgoing transfer processing.

`synergy_getSecurityAlerts`

```
synergy_getSecurityAlerts(account)
```

Returns protocol and wallet-plane security alerts relevant to an account.

Access profile: Authority plane profile.

Parameters

- **`account`** *(string)* — Account address or UMA identity address.

Returns

Array of security alert objects with severity, category, related transaction or approval identifiers, and remediation guidance.

Security and execution rules

- Authority-plane method. Do not expose this surface on unauthenticated public node endpoints.
- Operations that change signing authority, recovery state, or high-value transfer posture may require high-tier keys, guardian proofs, delay windows, or all three.
- Raw hash signing and single-factor spending authority are outside the Synergy security model.